

1222 • 2022
800
ANNI



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Containerize a web application with Docker

Web Applications

Master Degree in Computer Engineering
Academic Year 2025/2026



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

Ornella Irrera

Intelligent Interactive Information Access (IIIA) Hub
Department of Information Engineering
University of Padua



Introduction



Web application development

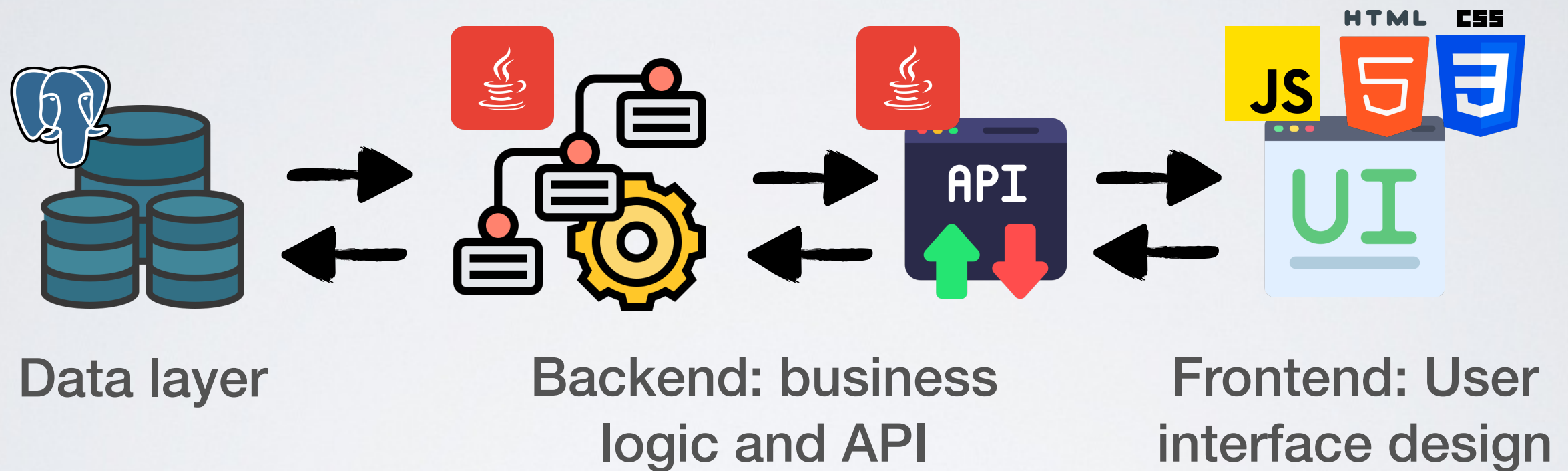


- A web application is composed of multiple technologies that must be correctly configured and integrated to work together.

Web application architecture



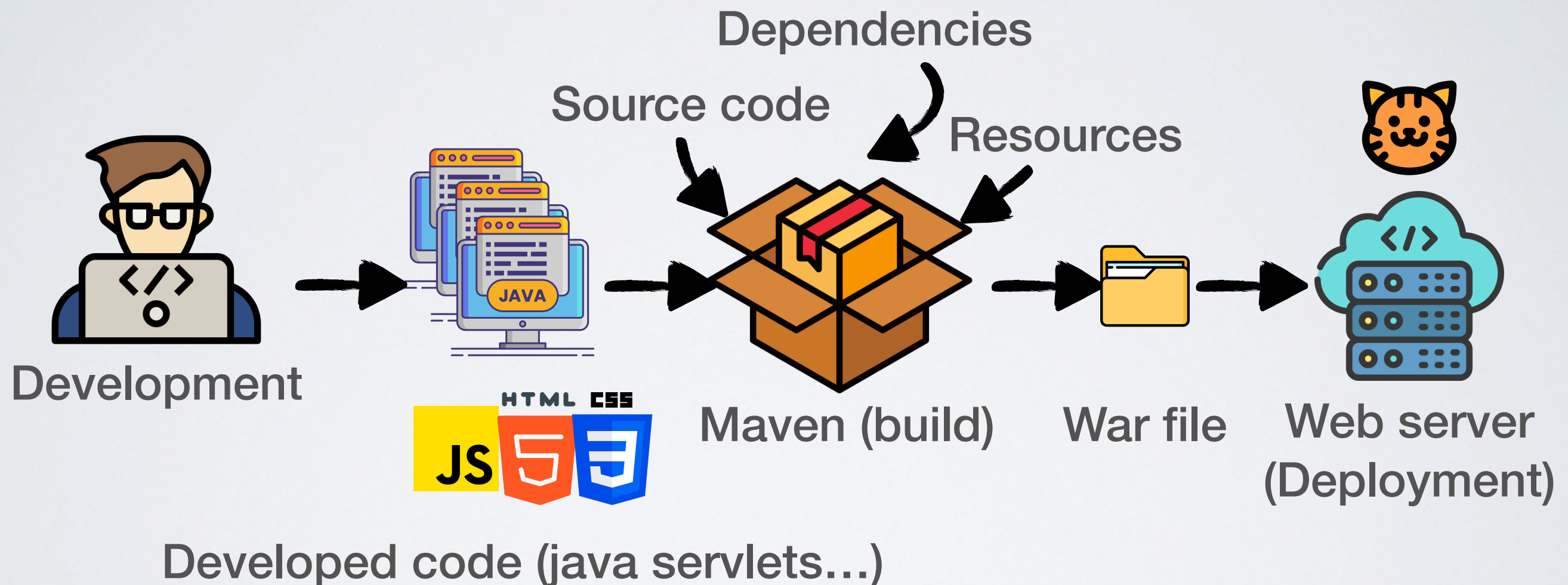
- A web application is composed of multiple technologies that must be correctly configured and integrated to work together.



Web application life cycle



- The development of a web application is a multi-step process involving **coding**, **building**, and **deployment**.





Web application development



- ⚠ Web applications are complex systems that require compatibility across multiple layers.
- ⚠ Even small version mismatches between development, build, and runtime environments can cause application failures.

Local deployment

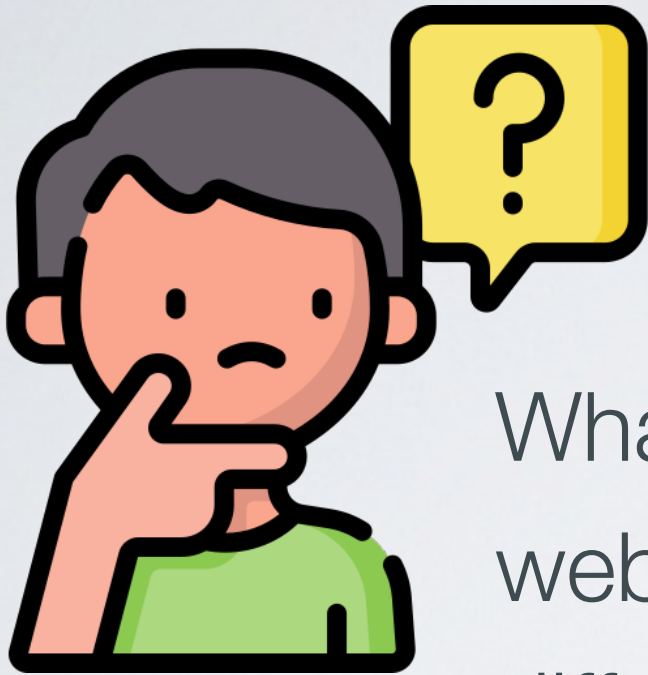


As a result, running a web application locally depends on the correct configuration of all its components.

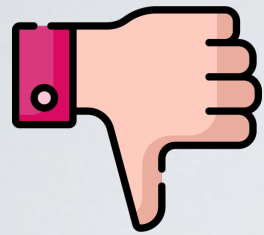


Running and maintaining the application **locally** is usually straightforward, as you have full control over the data, backend, and frontend layers.

Problem



What would happen if you have to deploy your web application on a new server that has different configurations, for example different PostgreSQL or java versions than yours?



Adapting your code to a different environment is extremely time-consuming and requires code and dependencies modifications



We need a solution **independent of the environment** where the web app is deployed



Is Maven enough?



- **No.** Maven helps manage Java projects by standardizing the build, testing, and packaging processes. However, it does not manage the deployment environment.
- Therefore, compatibility between components such as Tomcat must still be ensured.

Containerization



Containerization



- **Containerization** involves packaging an application into an isolated and self-sufficient execution environment that ensures consistent behavior across different platforms.



Containerization



- This is useful not only when you have to deploy a web application, but in a wide range of scenarios.
- Scenario 1: You have Python3.8 installed in your machine and you need to run some code in python3.11
- Scenario 2: Your OS is Windows and you want to use some tools which can be installed exclusively on Linux.

Linux
Python 3.11
Java21

Windows
Python 3.6
Java11

- Containerization creates isolated environments which are **independent** of your host machine.

Docker



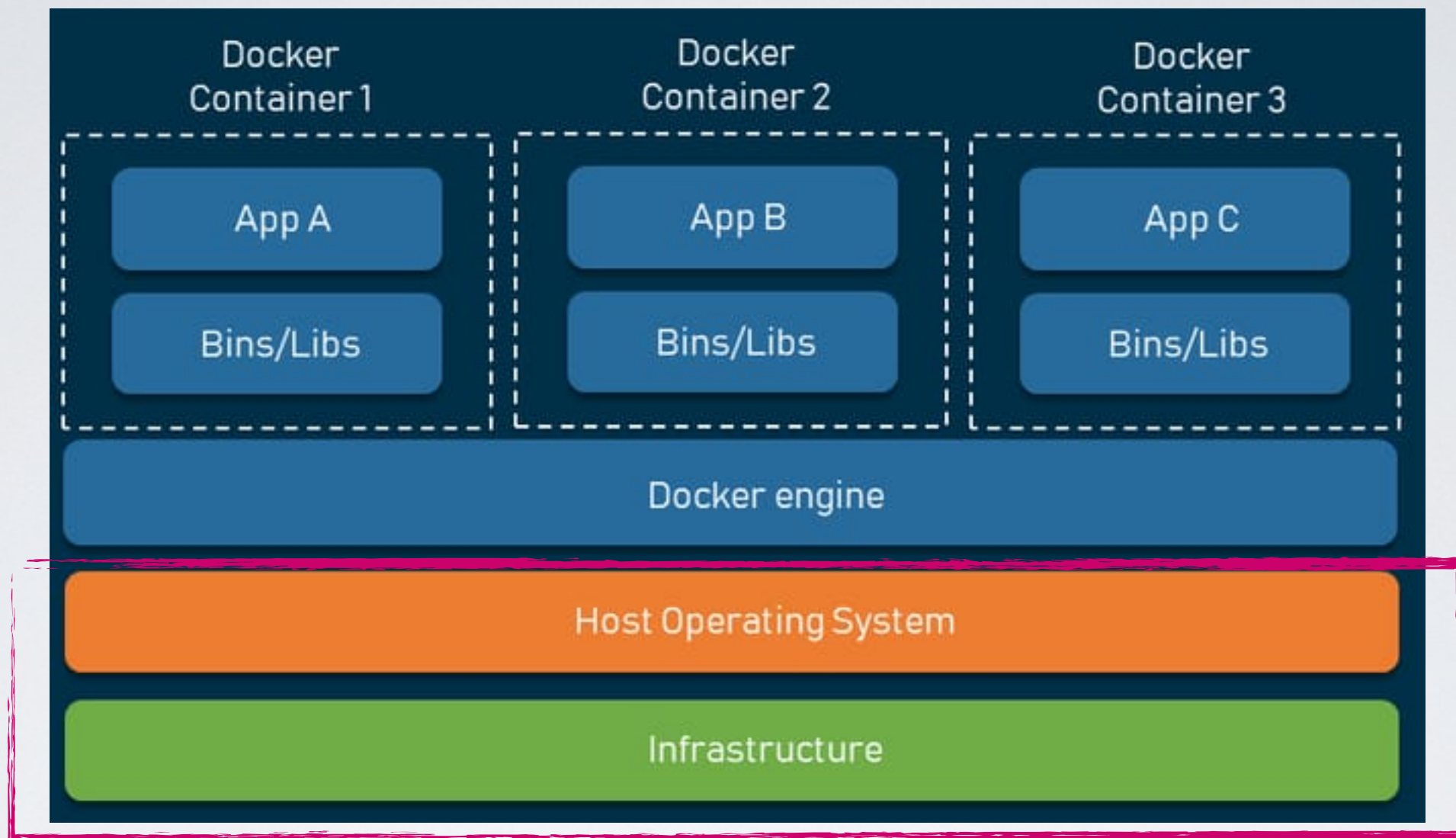
What is Docker?



- Docker is an open platform for developing, shipping, and running applications. It allows you to separate applications from the underlying infrastructure, enabling faster software delivery.
- Docker allows you to package and run applications in isolated environments called containers. This isolation makes it possible to run **multiple containers simultaneously** on a single host.

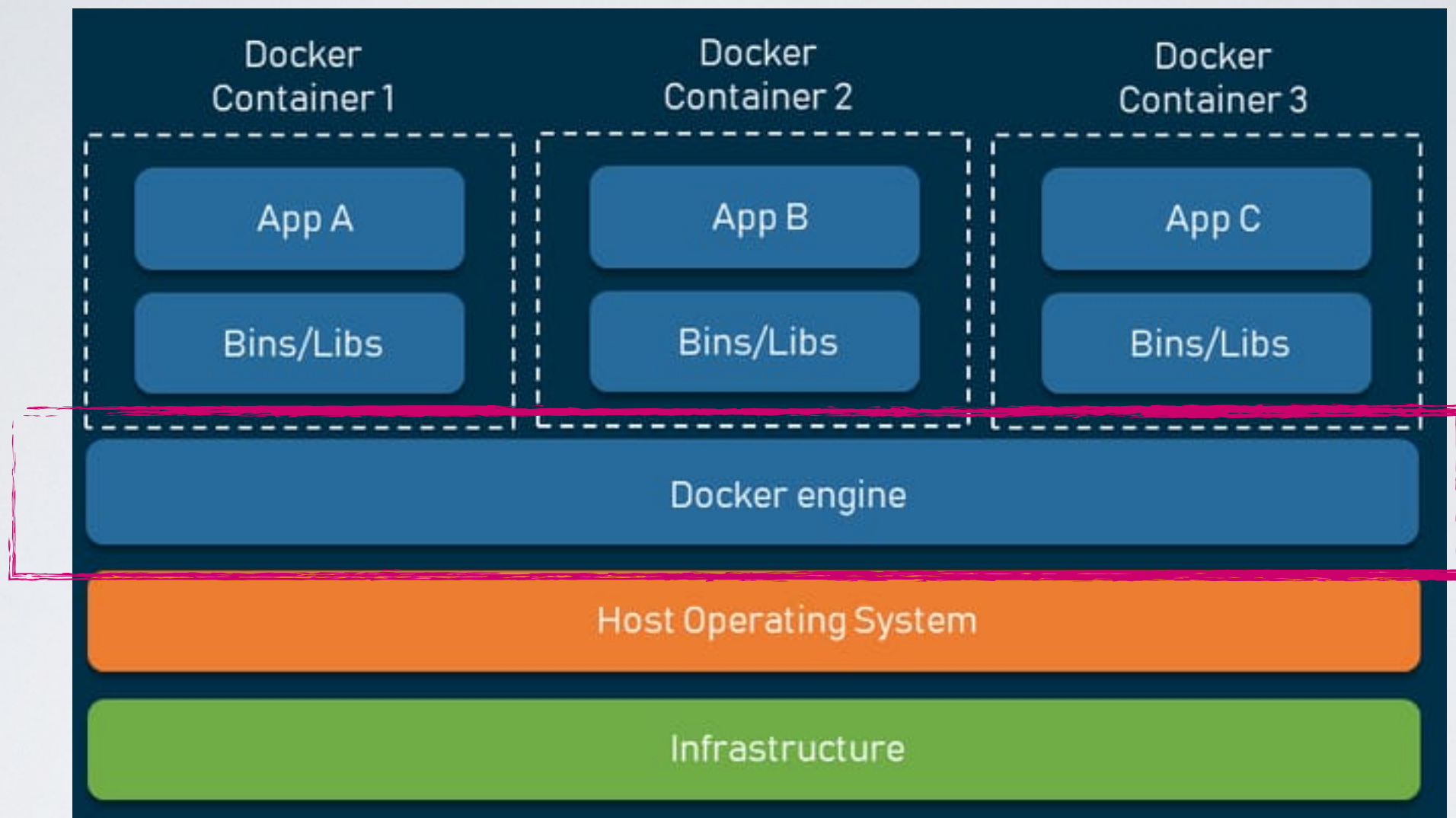
Documentation is available here: <https://docs.docker.com/get-started/overview/>

Docker: an overview



Where docker runs. It is the host server which typically has its own operating system.

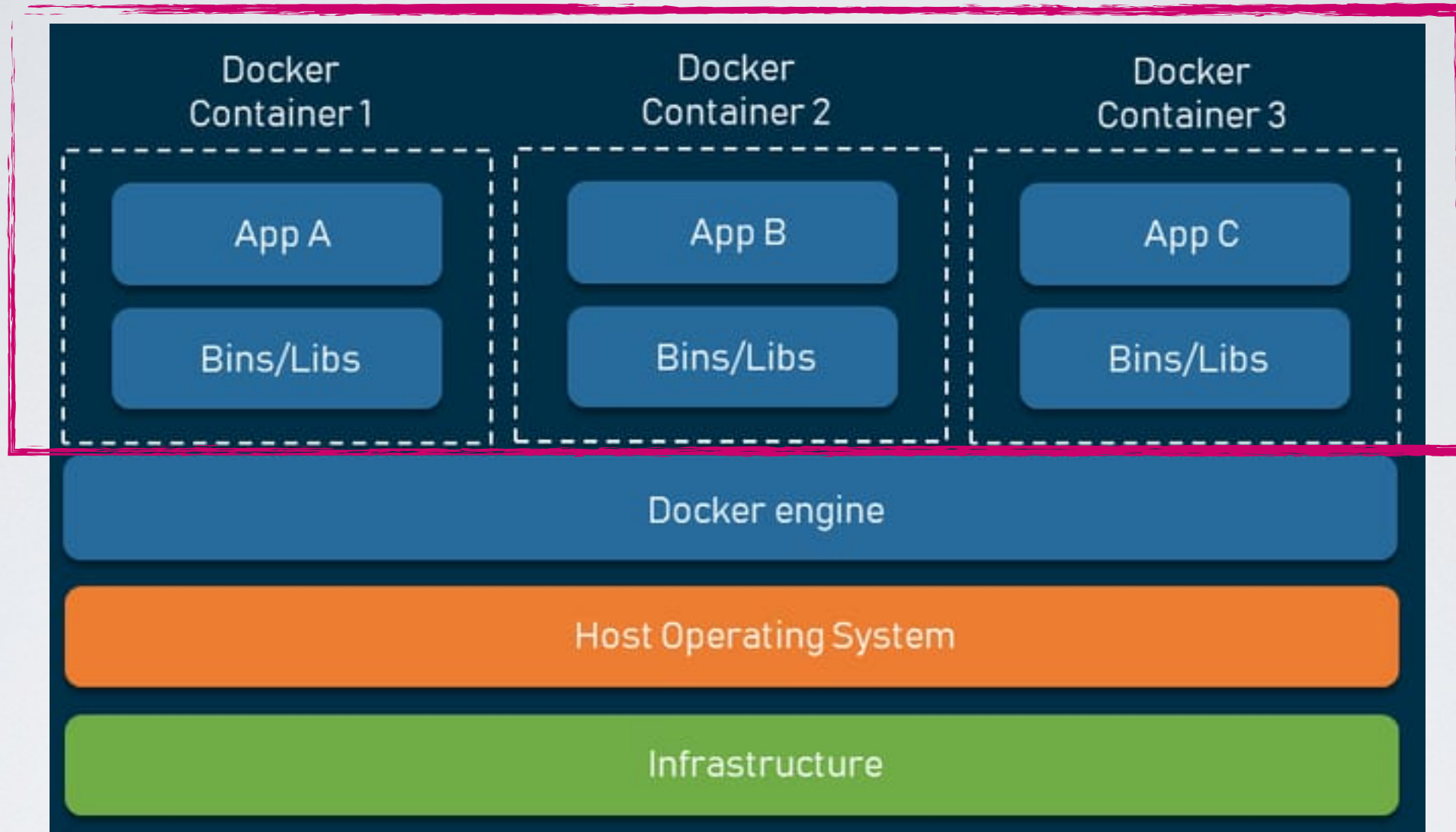
Docker: an overview



The docker engine, or more in general container engine, is a software component responsible for running and managing containers on a host machine.

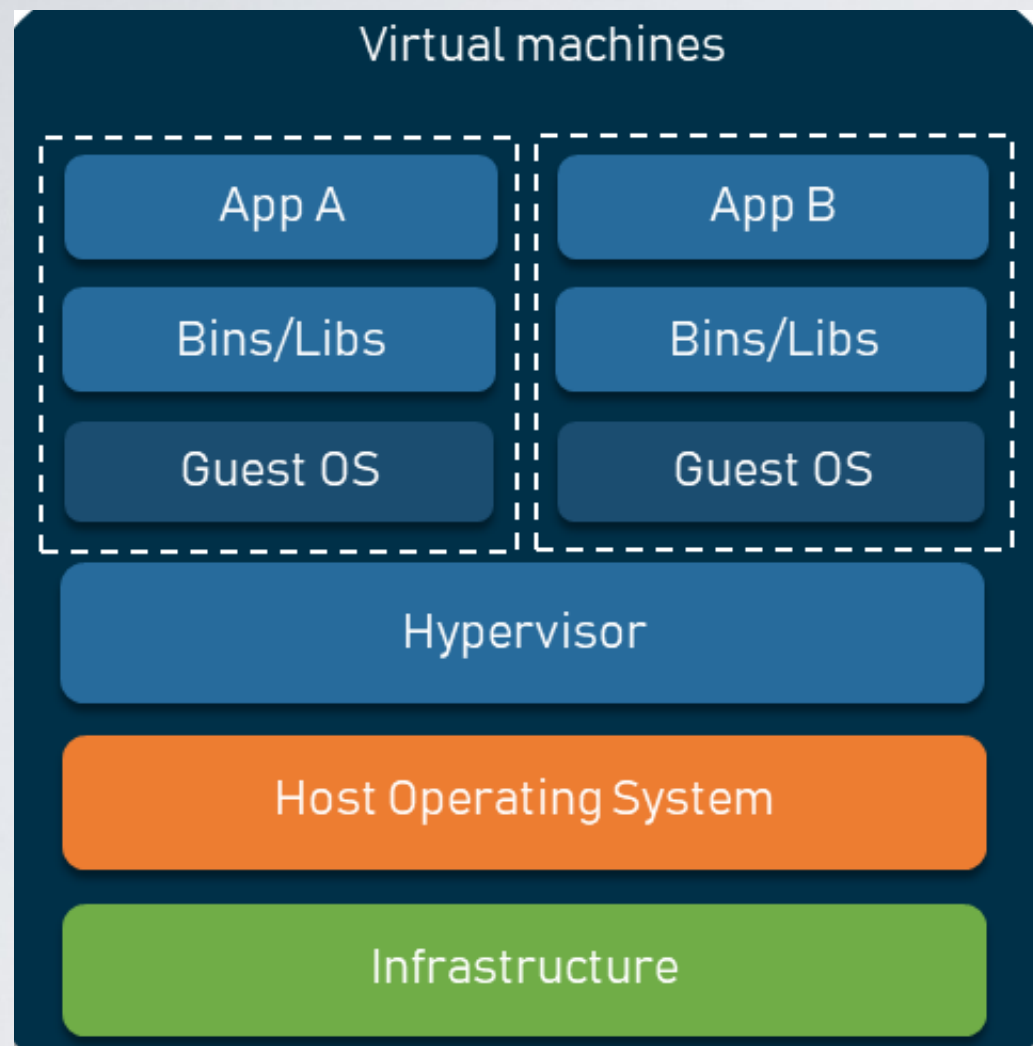
It provides the necessary functionality to create, start, stop, and manage containers, allowing applications to run in isolated environments.

Docker: an overview



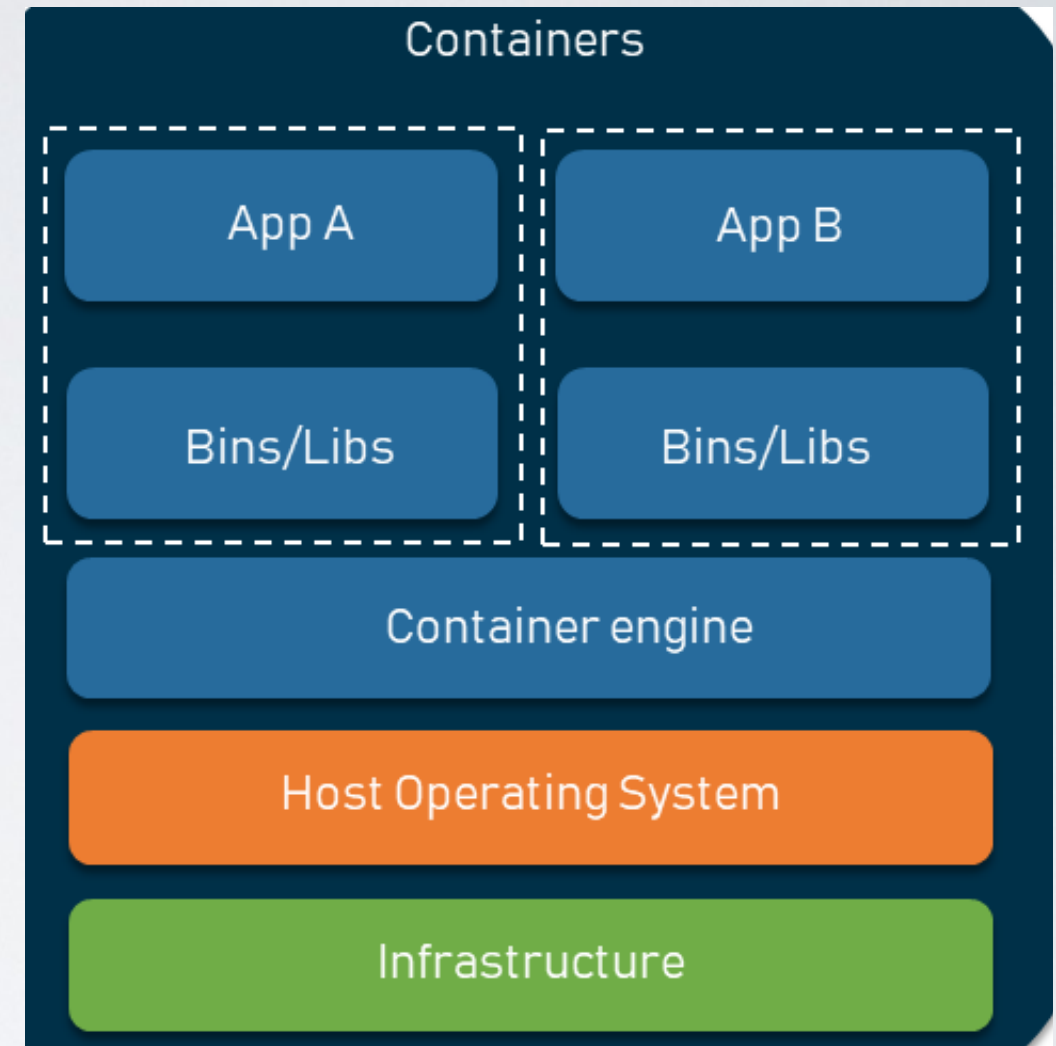
Isolated containerized applications. Each container has its own dependencies, its own libraries and are all independent of the underlying infrastructure and OS.

Containers vs VMs



Virtual machines are more isolated.

However, they are slower, difficult to be replicated, resource intensive.



Containers are: portable, lightweight, fast, easy deployment.

They all share the same kernel.



Docker main features



- **Portability** as containers can be easily run on any system that supports Docker, regardless of the underlying infrastructure or operating system.
- **Efficiency** as containers are lightweight and share the host system resources, resulting in different resources utilization.
- **Scalability** as applications can be deployed as multiple containers.
- **Isolation** as Docker containers provide process-level isolation, ensuring that applications and their dependencies run without interfering with each other or the host system.



Docker — the most important components

- The Docker objects collaborate together to run, access and manage the applications containers. The essential objects are the following:
 - **Images**
 - **Dockerfiles**
 - **Container**
 - **Volumes**
 - **Services**
 - **Networks**
- Other objects are **plugins, registries**



Docker images



- A Docker image is a read-only template used to create containers. It contains the libraries, dependencies, and instructions required for an application to run.
- Docker images are **immutable** and are composed of multiple layers. Each layer represents filesystem changes such as adding, removing, or modifying files.
- For example, you can start from a Python 3.11 base image, install required packages, and clone a repository, with each step creating a new layer.



Dockerfiles



- How can we create an image?



Dockerfiles



- To build user-defined Docker images, users create a Dockerfile, which is a text file that describes how the image should be built using a simple and declarative syntax.
- Dockerfiles contain instructions about dependencies, application configuration, port exposure, and commands required to run the application.



Dockerfiles



```
FROM nvidia/cuda:12.4.1-cudnn-runtime-ubuntu22.04

RUN apt-get update && apt-get install -y \
    python3.10 \
    python3.10-dev \
    python3-pip \
    git \
    curl \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

RUN update-alternatives --install /usr/bin/python python /usr/bin/python3.10 1

RUN python -m pip install --upgrade pip

WORKDIR /code/src
ENV PYTHONPATH=/code
COPY . /code
COPY requirements.txt .

# torch 2.10.0 GPU
RUN pip install --no-cache-dir -r requirements.txt
RUN pip install --no-cache-dir dgl -f https://data.dgl.ai/wheels/cu124/repo.html
RUN pip install torch==2.10.0 --index-url https://download.pytorch.org/whl/cu124

ENV PATH="/root/.local/bin:${PATH}"

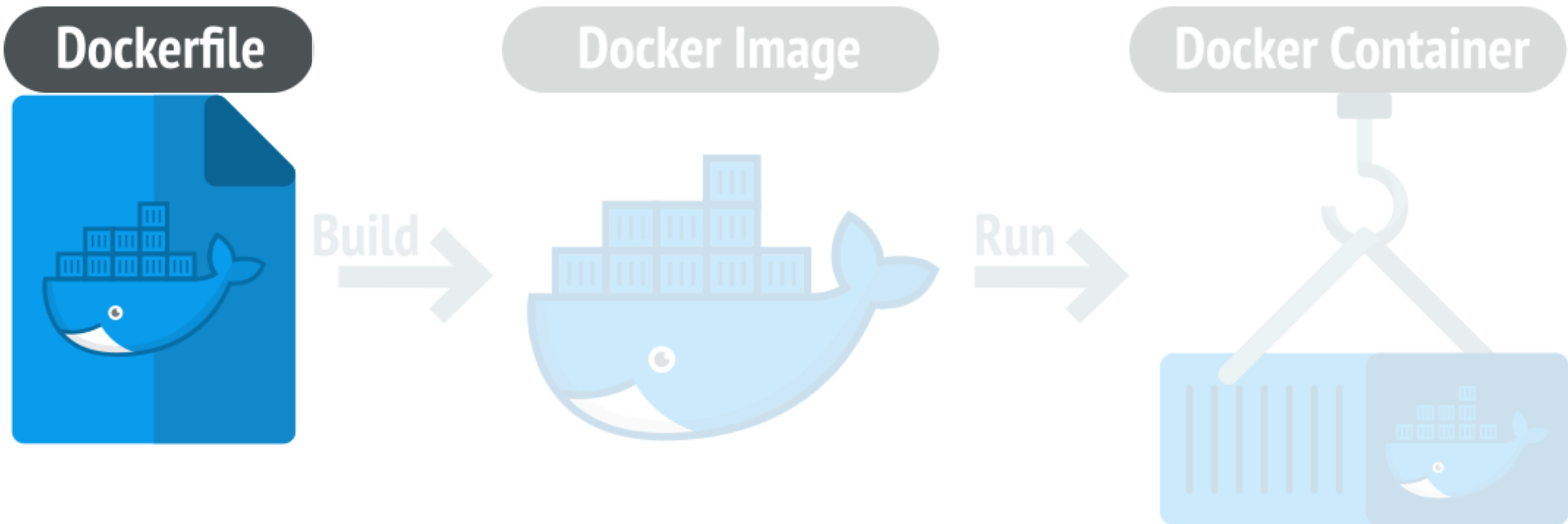
CMD ["/bin/bash"]
```



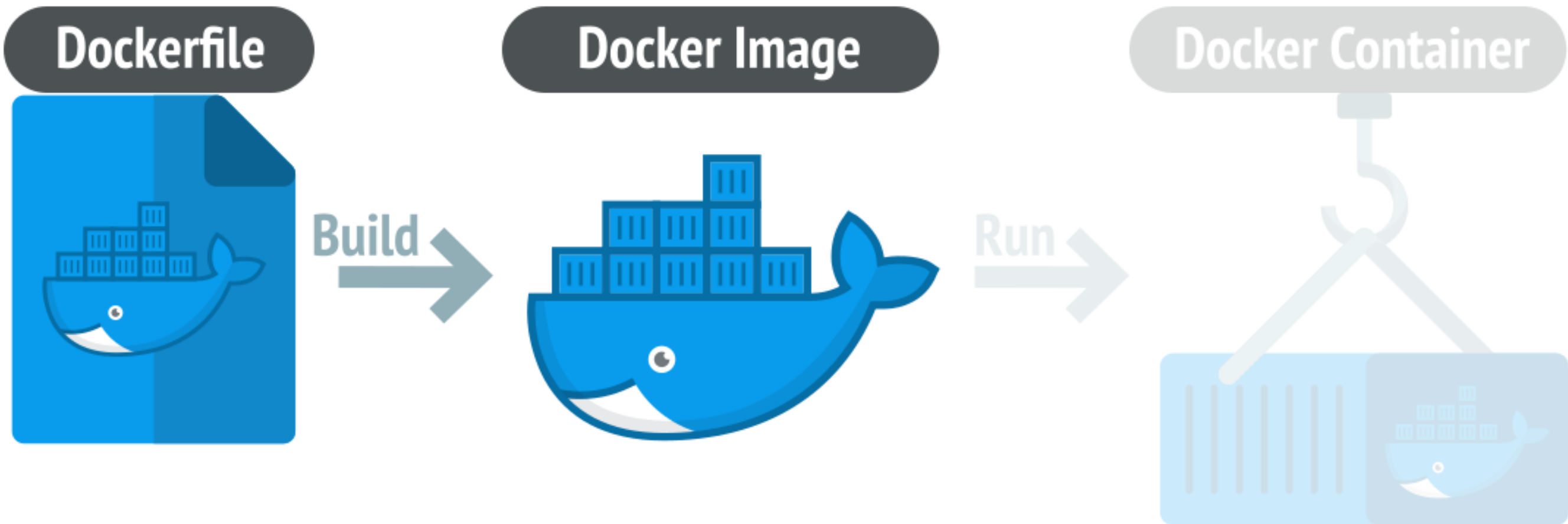
Docker container



- A Docker **container** is a lightweight and isolated runtime environment used to run applications.
- Containers are created from Docker images and are managed by the Docker Engine.
- Multiple containers can run on the same host using the same image without interfering with each other.
- Containers have a writable filesystem layer, but the underlying image remains immutable.

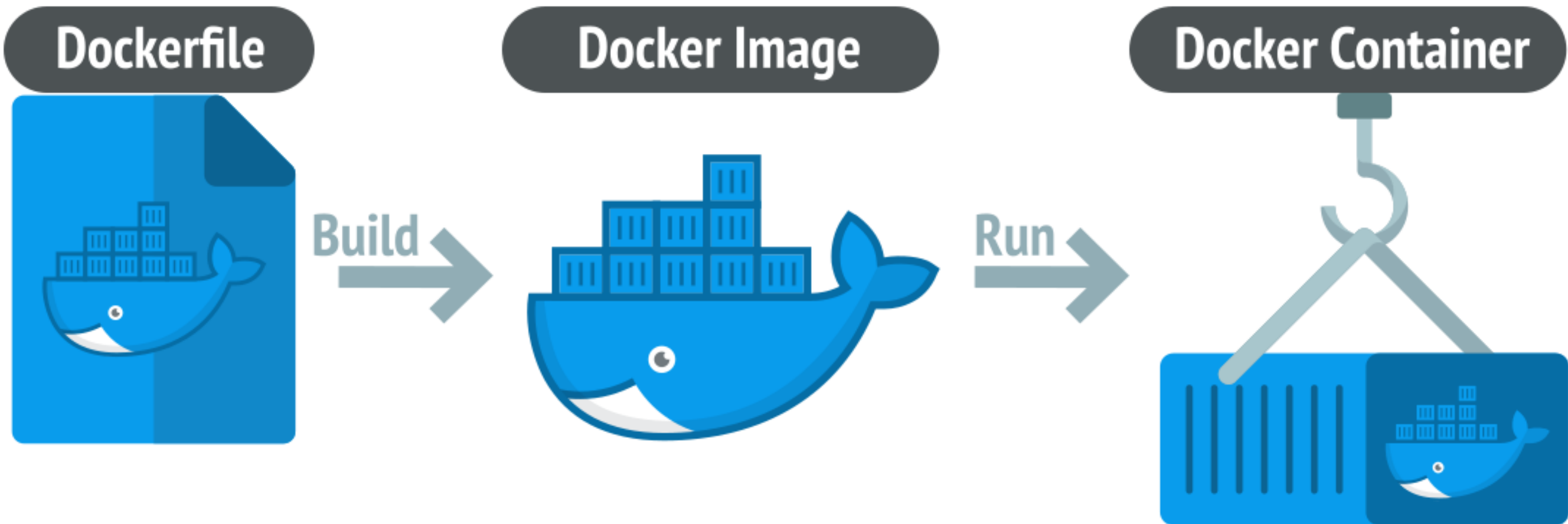


A Docker image is built based on instructions defined in a Dockerfile, which specifies the steps to create the image.



Images are created using the Docker build process, where the Docker Engine reads the Dockerfile and executes the instructions to build the image layer by layer.

Each layer represents a specific step in the build process.



When a Docker image is built, it is possible to create an instance of it called a container, which runs the application using the resources of the host machine.

Multiple containers can be created and run from the same image, each with its own isolated execution environment and state.



Docker volumes

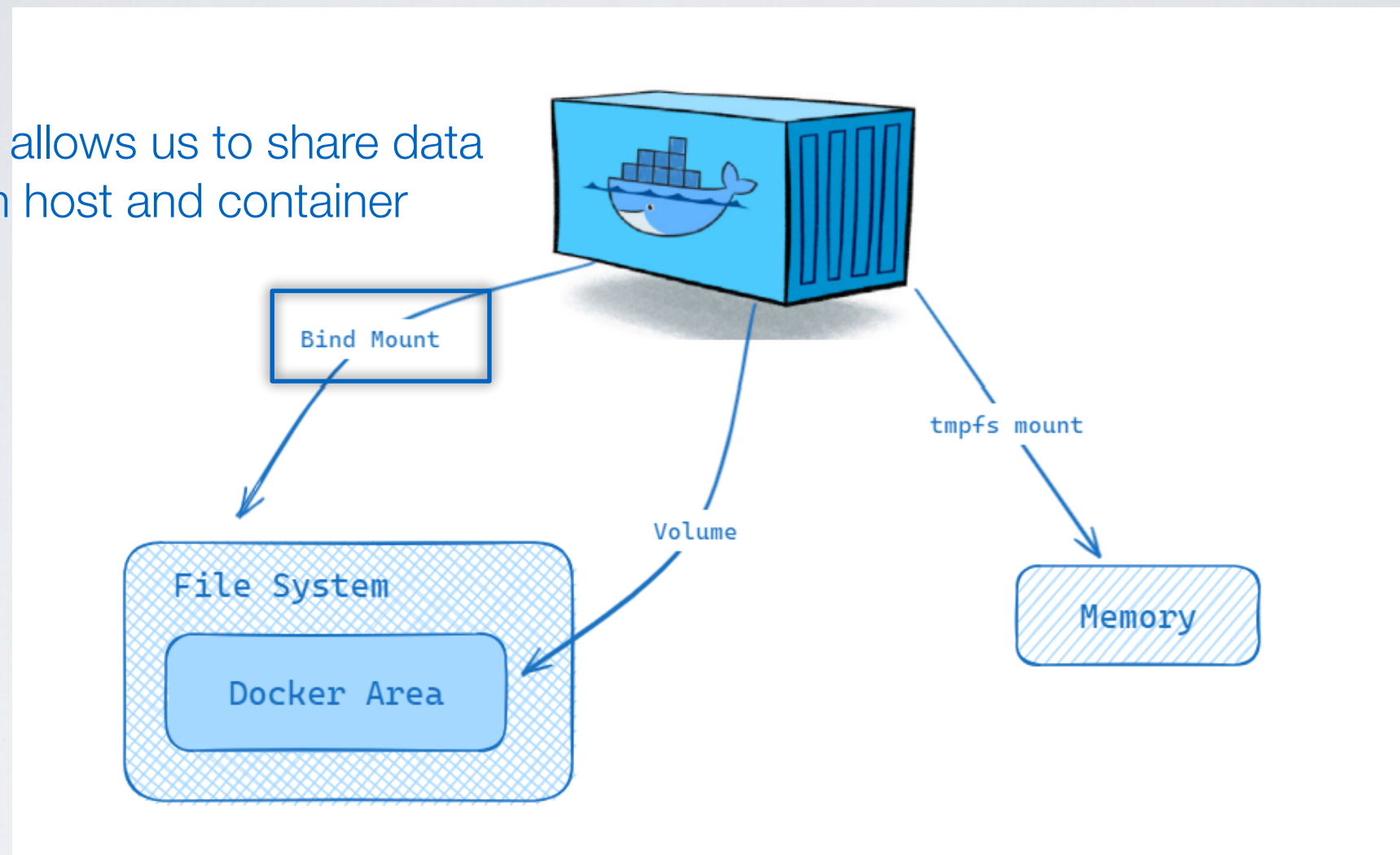


- In Docker, a volume is a mechanism for persistently storing data generated by and accessed by the docker containers.
- A Docker volume is a directory or file stored outside the container's writable layer, allowing data to persist even when containers are stopped or removed.
- It can be used to store databases, configuration files, logs, and any other data that needs to persist beyond the lifecycle of a container.

Docker volumes



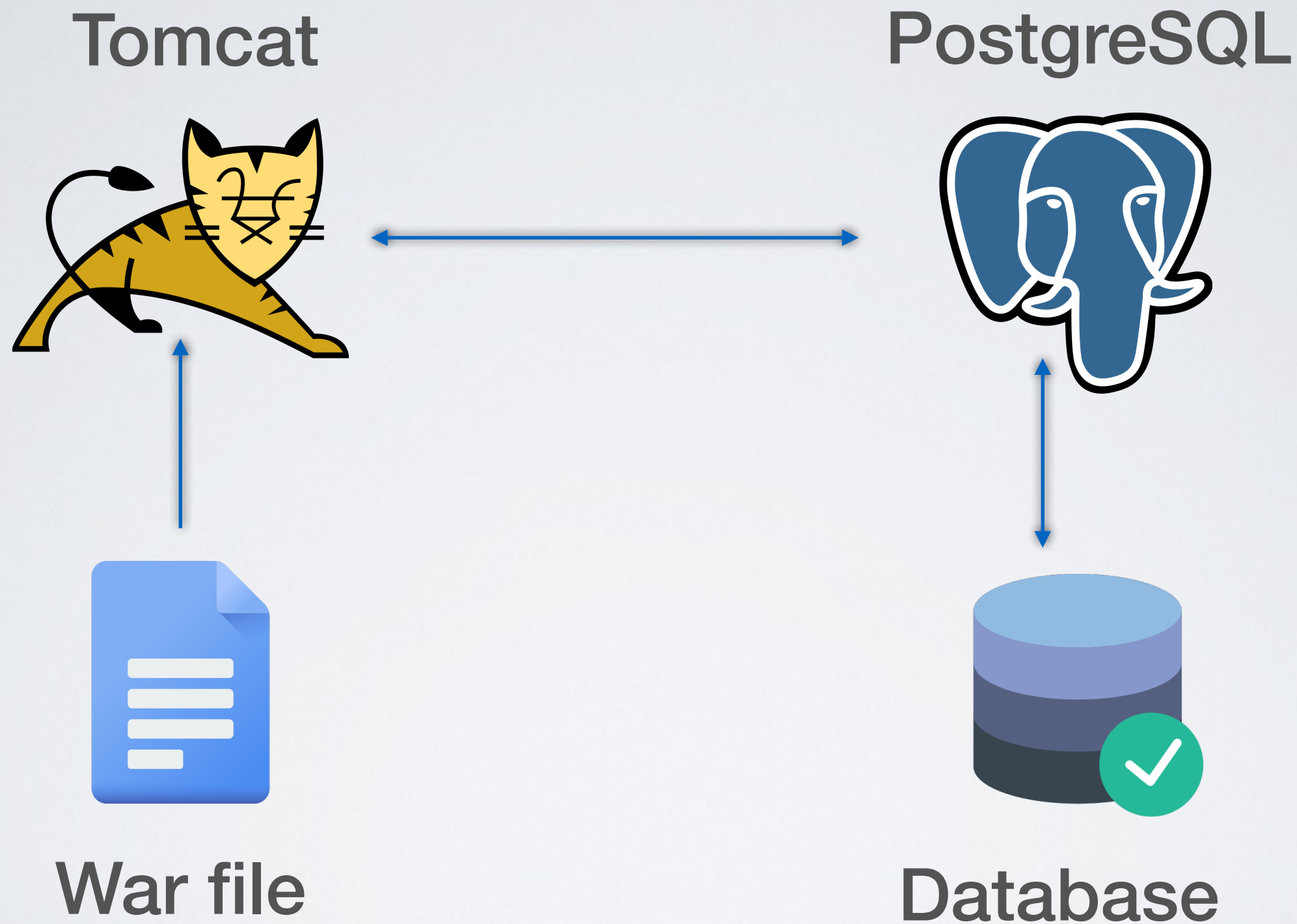
Bind mount allows us to share data between host and container



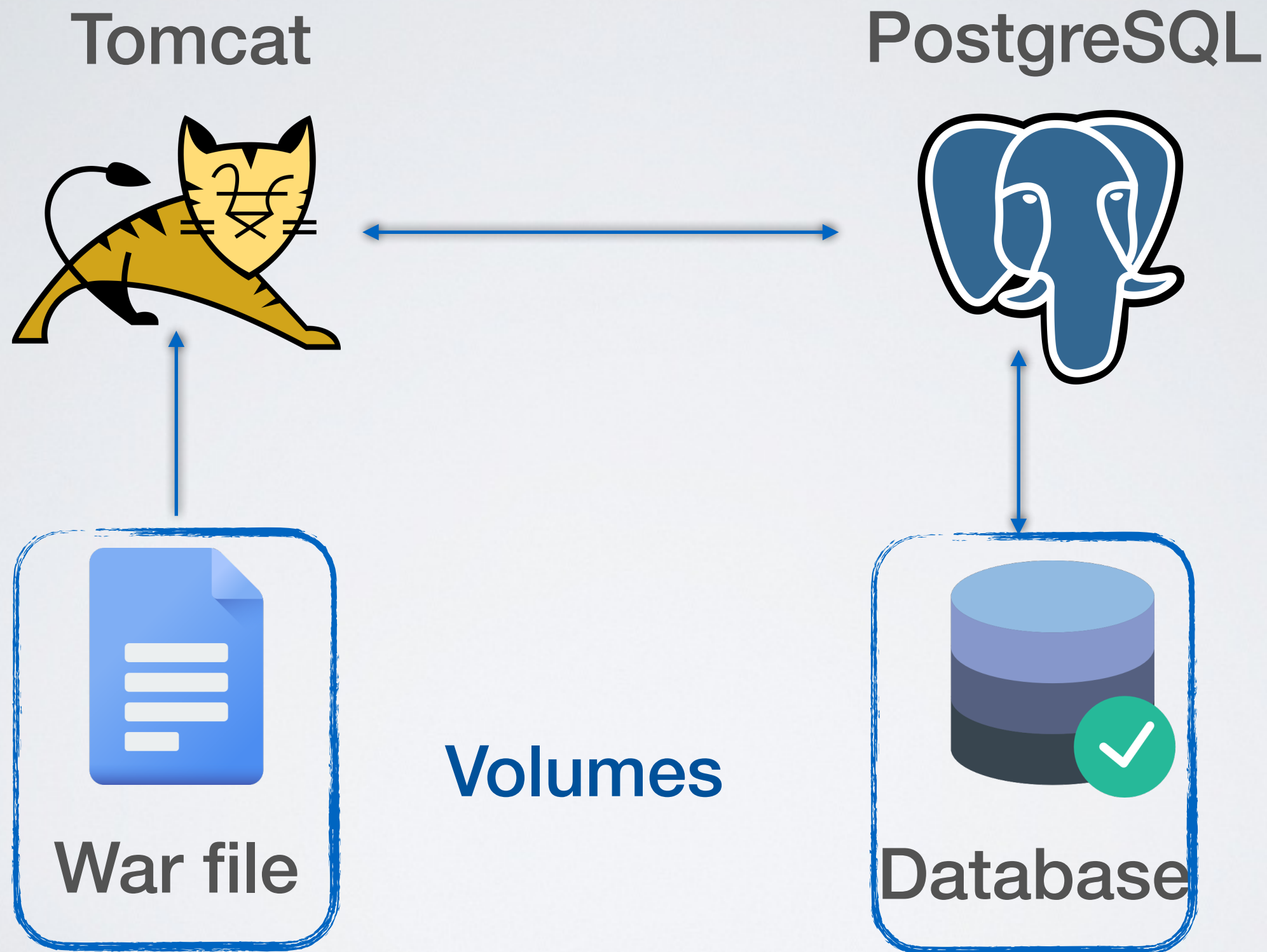
- Multiple containers can mount the same volumes
- Volumes persist **independently** of the lifecycle of the container
- Volumes can be attached and detached from the containers as needed



Your web application



Your web application





Problem



- Web applications are complex systems composed of several components, such as the **database**, and **web server**, which must communicate by exchanging requests, responses, and processed data.
- How can we assemble and manage all these components using Docker?

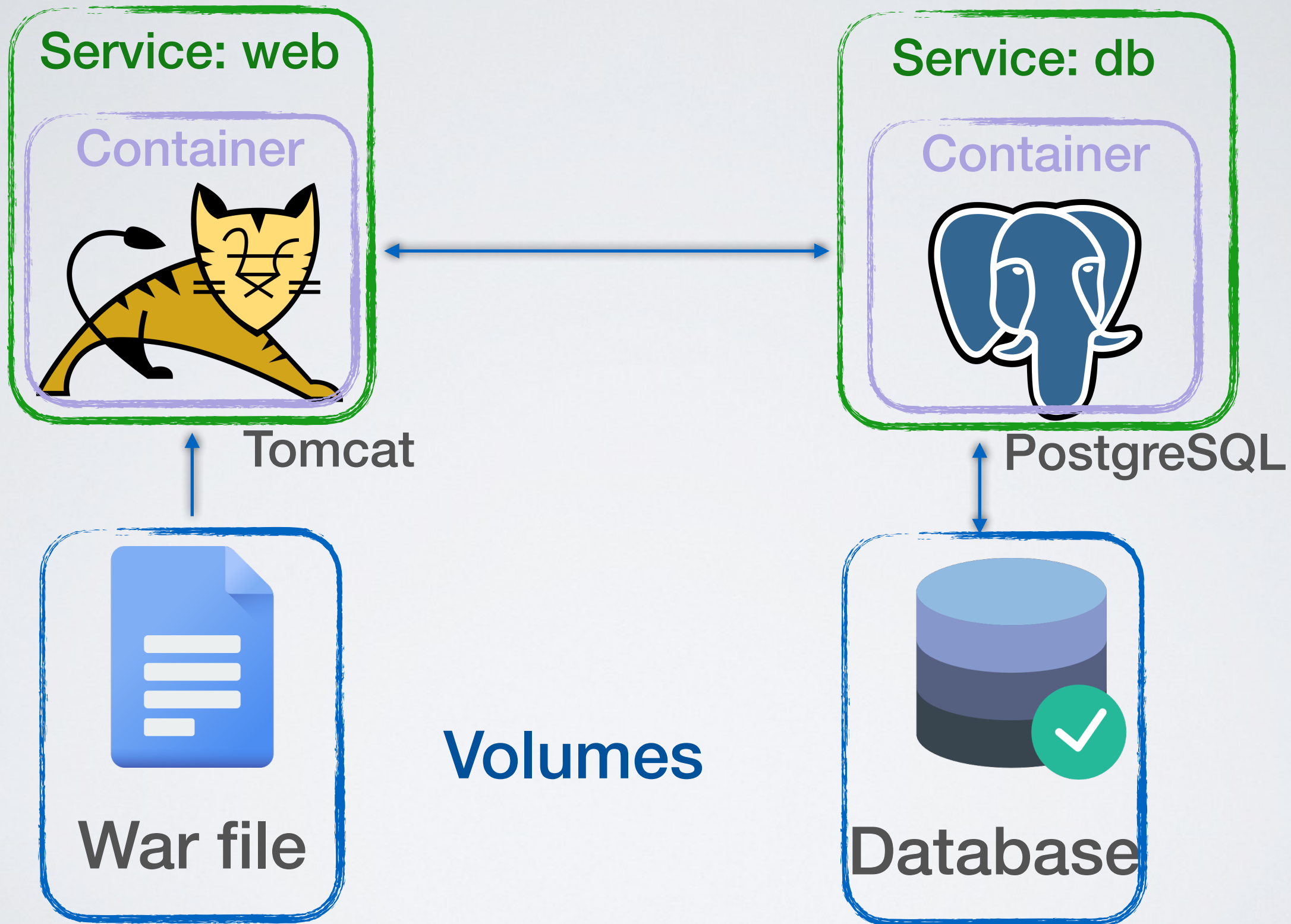


Docker services



- A **service** represents a component of an application, such as a web server (e.g., Tomcat) or a database (e.g., PostgreSQL)
- A service is typically based on a single Docker image and can be scaled by running multiple container replicas that provide the same functionality.
- Services allow you to manage and deploy different application components as independent units.

Your web application



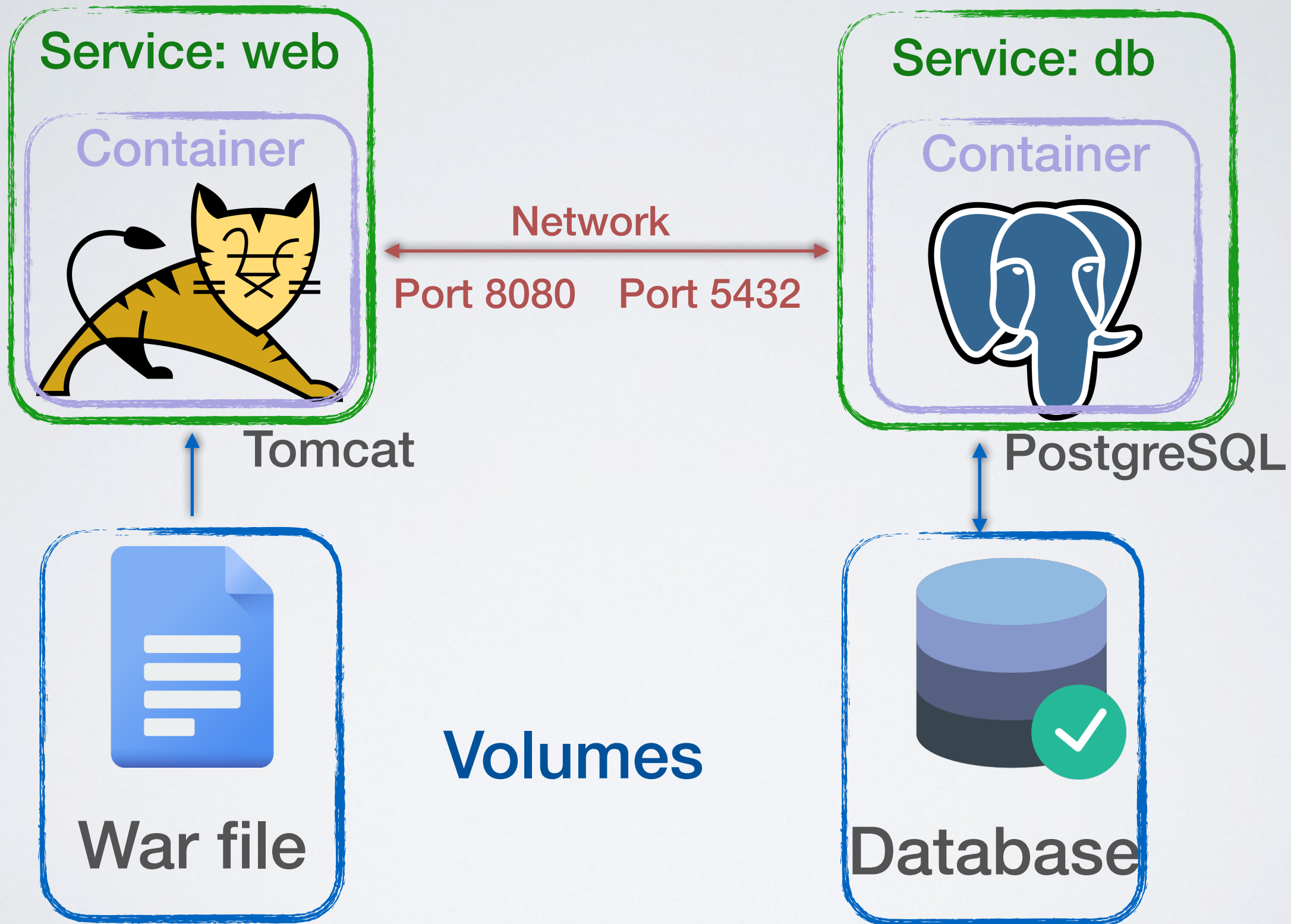


Docker networks



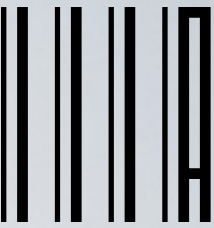
- Docker containers are all isolated and they cannot communicate: hence, given two containers, A and B, exposing different ports, A cannot communicate with B.
- On the other hand, we can reach A and B from outside the container, if the ports are properly specified.
- **Docker networks are the solution to let containers communicate.**
- Only the containers belonging to the same network can communicate. When a container is instantiated, the network it belongs to should be specified.

Your web application





Docker compose

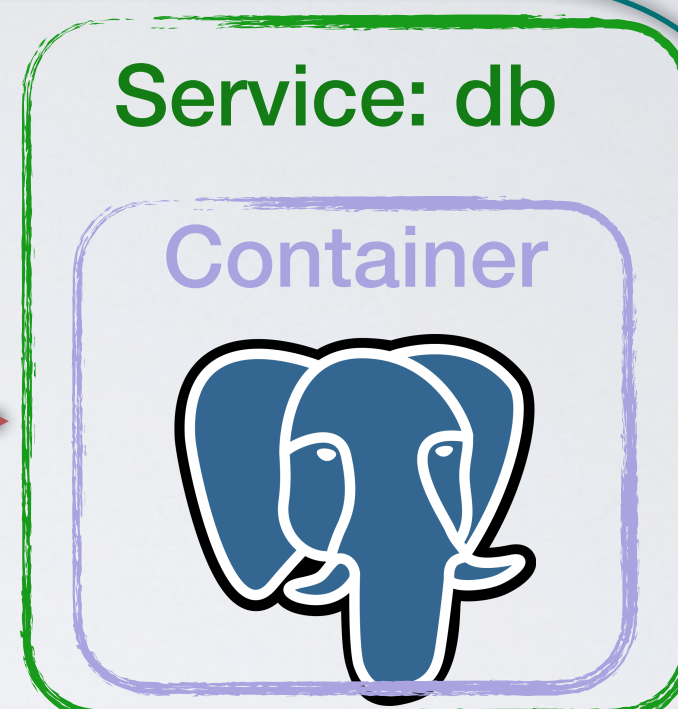


- Docker Compose is a tool that simplifies the management and deployment of multi-containers applications —our web application is multi container!!
- It allows you to define and configure multiple Docker services, networks and volumes as a single application using a YAML file.
- Typical use-case: you have a web application. Each of these component is treated as a separate container which is part of the same web application.

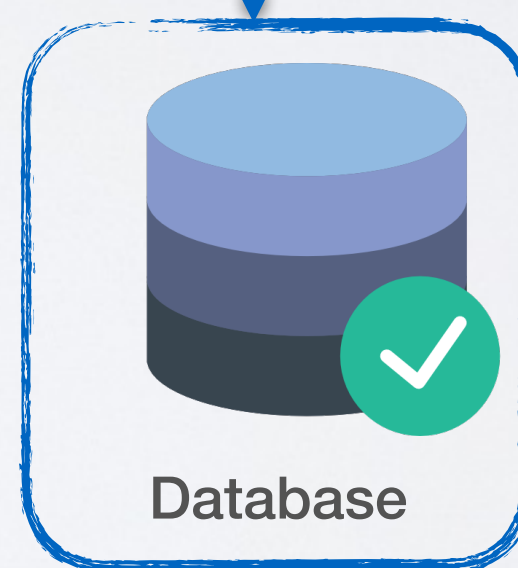
Your web application



Tomcat



PostgreSQL



Network

Port 8080 Port 5432



Volumes

Docker compose



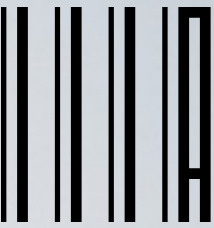
Summing up



- We developed our entire web application which includes the backend developed in java, the frontend developed in css, html and js, and a postgresSQL database.
- Maven produces a .war file which can be deployed on a web server (Tomcat).
- We want to deploy our web application on multi-containerized environment which implements two different services:
 - Tomcat
 - PostgreSQL
- This allows you to deploy your war file you created locally on the version of tomcat and postgresSQL you prefer, without having to adapt your web application to the hosting infrastructure.



Summing up



- All the services in a multi-containerized environment are described in the docker-compose.yml file.
- Docker Compose creates a network for your application, and each service is connected to that network. Services can communicate with each other **using their service names as hostnames**.
- We need to understand which are the services we need to deploy our web application: in this case we have tomcat and PostgreSQL.
- We need to specify some dependencies: establishing a **hierarchies** of dependencies allows to determine the order in which the services are executed.
- Each service relies on a specific image, ports, volumes, environment variables e developed our entire web application which includes the backend developed in java, the frontend developed in css, html and js, and a postgresSQL database.

Hands on



Install



- You need to install docker and docker compose following the guidelines in the documentation
- Docker: <https://docs.docker.com/get-docker/>
- Docker compose: <https://docs.docker.com/compose/>



The docker compose file

```
services:
```

```
  web:
```

```
    image: tomcat:10
```

```
    ports:
```

```
      - "8080:8080"
```

```
    depends_on:
```

```
      db:
```

```
        condition: service_healthy
```

```
    volumes:
```

```
      - ./crane.war:/usr/local/tomcat/webapps/crane.war
```

```
  db:
```

```
    image: postgres
```

```
    ports:
```

```
      - "5432:5432"
```

```
    environment:
```

```
      - POSTGRES_PASSWORD=postgres
```

```
      - POSTGRES_USER=postgres
```

```
    volumes:
```

```
      - ./crane.sql:/docker-entrypoint-initdb.d/init.sql
```

```
      - ./data/db:/var/lib/postgresql/data
```

```
    healthcheck:
```

```
      test: [ "CMD-SHELL", "pg_isready -U postgres" ]
```

```
      interval: 5s
```

```
      timeout: 10s
```

```
      retries: 50
```



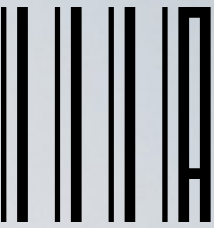
The docker compose file

services:

In the docker-compose.yml file we define the services needed to run our web application.

```
web:
  image: tomcat:10
  ports:
    - "8080:8080"
  depends_on:
    db:
      condition: service_healthy
  volumes:
    - ./crane.war:/usr/local/tomcat/webapps/crane.war

db:
  image: postgres
  ports:
    - "5432:5432"
  environment:
    - POSTGRES_PASSWORD=postgres
    - POSTGRES_USER=postgres
  volumes:
    - ./crane.sql:/docker-entrypoint-initdb.d/init.sql
    - ./data/db:/var/lib/postgresql/data
  healthcheck:
    test: [ "CMD-SHELL", "pg_isready -U postgres" ]
    interval: 5s
    timeout: 10s
    retries: 50
```



The docker compose file

services:

web:

image: tomcat:10

ports:

– "8080:8080"

depends_on:

db:

condition: service_healthy

volumes:

– ./crane.war:/usr/local/tomcat/webapps/crane.war

web and **db** are the names of the two services we are defining.

These names are useful not only to identify the individual services, but also to establish connections and let the services communicate.

db:

image: postgres

ports:

– "5432:5432"

environment:

– POSTGRES_PASSWORD=postgres

– POSTGRES_USER=postgres

volumes:

– ./crane.sql:/docker-entrypoint-initdb.d/init.sql

– ./data/db:/var/lib/postgresql/data

healthcheck:

test: ["CMD-SHELL", "pg_isready -U postgres"]

interval: 5s

timeout: 10s

retries: 50



The docker compose file

services:

Web and db run two containers, hence, we need to specify the image they refer to. The image can be found on a registry (as in this case), and we need only to specify the image name; alternatively, it might be defined in a dockerfile and we have to specify the location of the dockerfile.

web:

image: tomcat:10

ports:

- "8080:8080"

depends_on:

db:

condition: service_healthy

volumes:

- ./crane.war:/usr/local/tomcat/webapps/crane.war

db:

image: postgres

ports:

- "5432:5432"

environment:

- POSTGRES_PASSWORD=postgres

- POSTGRES_USER=postgres

volumes:

- ./crane.sql:/docker-entrypoint-initdb.d/init.sql

- ./data/db:/var/lib/postgresql/data

healthcheck:

test: ["CMD-SHELL", "pg_isready -U postgres"]

interval: 5s

timeout: 10s

retries: 50



The docker compose file

services:

To access the service from within the container:

- the hostname is the name of the service (web)
- The port is the container port (8080)
- <http://web:8080/> to access web from the Postgres service container.

web:

image: tomcat:10

ports:

- "8080:8080"

8080:8080 is **host_port:container:port**

depends_on:

db:

condition: service_healthy

volumes:

- ./crane.war:/usr/local/tomcat/webapps/crane.war

db:

image: postgres

ports:

- "5432:5432"

environment:

- POSTGRES_PASSWORD=postgres

- POSTGRES_USER=postgres

volumes:

- ./crane.sql:/docker-entrypoint-initdb.d/init.sql

- ./data/db:/var/lib/postgresql/data

healthcheck:

test: ["CMD-SHELL", "pg_isready -U postgres"]

interval: 5s

timeout: 10s

retries: 50



The docker compose file

services:

A service allows us to specify environment variables.

web:

These information will also be used in the context.xml file!!

```
image: tomcat:10
ports:
  - "8080:8080"
depends_on:
  db:
    condition: service_healthy
volumes:
  - ./crane.war:/usr/local/tomcat/webapps/crane.war
```

db:

```
image: postgres
ports:
  - "5432:5432"
environment:
  - POSTGRES_PASSWORD=postgres
  - POSTGRES_USER=postgres
volumes:
  - ./crane.sql:/docker-entrypoint-initdb.d/init.sql
  - ./data/db:/var/lib/postgresql/data
healthcheck:
  test: [ "CMD-SHELL", "pg_isready -U postgres" ]
  interval: 5s
  timeout: 10s
  retries: 50
```

The docker compose file

services:

web:

image: tomcat:10

ports:

– "8080:8080"

depends_on:

db:

condition: service_healthy

volumes:

– ./crane.war:/usr/local/tomcat/webapps/crane.war

In order to define volumes we need to specify the (relative) path to the file in the hosting machine.

After the «:» we specify where this file will be mounted inside the container.

db:

image: postgres

ports:

– "5432:5432"

environment:

– POSTGRES_PASSWORD=postgres

– POSTGRES_USER=postgres

volumes:

– ./crane.sql:/docker-entrypoint-initdb.d/init.sql

– ./data/db:/var/lib/postgresql/data

The docker-entrypoint-init-db.d directory is a convention to provide a mechanism for initializing a database when a container is first started.

healthcheck:

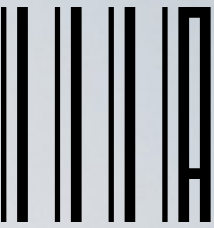
test: ["CMD-SHELL", "pg_isready -U postgres"]

interval: 5s

timeout: 10s

retries: 50

The crane.sql file contains the code to create the database. The tables and the data will be automatically created as soon as the container is instantiated for the first time.



The docker compose file

services:

web:

image: tomcat:10

ports:

– "8080:8080"

depends_on:

db:

condition: service_healthy

volumes:

– ./crane.war:/usr/local/tomcat/webapps/crane.war

db:

image: postgres

ports:

– "5432:5432"

environment:

– POSTGRES_PASSWORD=postgres

– POSTGRES_USER=postgres

volumes:

– ./crane.sql:/docker-entrypoint-initdb.d/init.sql

– ./data/db:/var/lib/postgresql/data

healthcheck:

test: ["CMD-SHELL", "pg_isready -U postgres"]

interval: 5s

timeout: 10s

retries: 50

Healthchecks allow us to define commands to check the status of a service.

test is the command to execute. In this case we want to check the connection status of a postgresQL server. Without this check we would try to establish the connection when postgres is not ready and this would lead to connection refused.

Interval is the interval at which the test is executed.

Timeout is the max amount of time the check has to finish

Retries is number of consecutive failures allowed



The docker compose file



services:

web:

image: tomcat:10

ports:

- "8080:8080"

depends_on:

db:

condition: service_healthy

volumes:

- ./crane.war:/usr/local/tomcat/webapps/crane.war

db:

image: postgres

ports:

- "5432:5432"

environment:

- POSTGRES_PASSWORD=postgres

- POSTGRES_USER=postgres

volumes:

- ./crane.sql:/docker-entrypoint-initdb.d/init.sql

- ./data/db:/var/lib/postgresql/data

healthcheck:

test: ["CMD-SHELL", "pg_isready -U postgres"]

interval: 5s

timeout: 10s

retries: 50

Dependencies establish the execution order . In this example, the db is initiated first; then, the web service is run as soon as the healthcheck is satisfied. depends_on in general does not wait a service to be fully initialized and healthy. It only waits for the dependent service to be running. To wait a service to be ready we need to establish healthchecks.



The docker compose file



services:

web:

image: tomcat:10

ports:

- "8080:8080"

depends_on:

db:

condition: service_healthy

volumes:

- ./crane.war:/usr/local/tomcat/webapps/crane.war

db:

image: postgres

ports:

- "5432:5432"

environment:

- POSTGRES_PASSWORD=postgres

- POSTGRES_USER=postgres

volumes:

- ./crane.sql:/docker-entrypoint-initdb.d/init.sql

- ./data/db:/var/lib/postgresql/data

healthcheck:

test: ["CMD-SHELL", "pg_isready -U postgres"]

interval: 5s

timeout: 10s

retries: 50

Dependencies establish the execution order . In this example, the db is initiated first; then, the web service is run as soon as the healthcheck is satisfied.

depends_on in general does not wait a service to be fully initialized and healthy. It only waits for the dependent service to be running. To wait a service to be ready we need to establish healthchecks.



Running



```
ornellairrera@MacBook-Pro-di-Ornella docker % docker-compose up
[+] Running 2/0
 ✓ Container docker-db-1 Created 0.0s
 ✓ Container docker-web-1 Created 0.0s
Attaching to docker-db-1, docker-web-1
docker-db-1 | The files belonging to this database system will be owned by user "postgres".
docker-db-1 | This user must also own the server process.
docker-db-1 |
docker-db-1 | The database cluster will be initialized with locale "en_US.utf8".
docker-db-1 | The default database encoding has accordingly been set to "UTF8".
docker-db-1 | The default text search configuration will be set to "english".
docker-db-1 |
docker-db-1 | Data page checksums are disabled.
docker-db-1 |
docker-db-1 | fixing permissions on existing directory /var/lib/postgresql/data ... ok
docker-db-1 | creating subdirectories ... ok
docker-db-1 | selecting dynamic shared memory implementation ... posix
docker-db-1 | selecting default max_connections ... 100
docker-db-1 | selecting default shared_buffers ... 128MB
docker-db-1 | selecting default time zone ... Etc/UTC
docker-db-1 | creating configuration files ... ok
docker-db-1 | running bootstrap script ... ok
docker-db-1 | performing post-bootstrap initialization ... ok
docker-db-1 | initdb: warning: enabling "trust" authentication for local connections
docker-db-1 | initdb: hint: You can change this by editing pg_hba.conf or using the option -A, or --auth-local and --auth-host, the next time you run initdb
b.
docker-db-1 | syncing data to disk ... ok
docker-db-1 |
docker-db-1 | Success. You can now start the database server using:
docker-db-1 |
docker-db-1 |     pg_ctl -D /var/lib/postgresql/data -l logfile start
docker-db-1 |
docker-db-1 | waiting for server to start....2023-05-30 09:33:07.039 UTC [84] LOG:  starting PostgreSQL 15.3 (Debian 15.3-1.pgdg110+1) on x86_64-pc-linux-g
nu, compiled by gcc (Debian 10.2.1-6) 10.2.1 20210110, 64-bit
docker-db-1 | 2023-05-30 09:33:07.042 UTC [84] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
docker-db-1 | 2023-05-30 09:33:07.083 UTC [87] LOG:  database system was shut down at 2023-05-30 09:33:03 UTC
docker-db-1 | 2023-05-30 09:33:07.137 UTC [84] LOG:  database system is ready to accept connections
docker-db-1 | done
docker-db-1 | server started
docker-db-1 |
docker-db-1 | /usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/init.sql
docker-db-1 | CREATE DATABASE
docker-db-1 | GRANT
docker-db-1 | You are now connected to database "ssadb" as user "postgres".
docker-db-1 | SET
docker-db-1 | SET
docker-db-1 | SET
docker-db-1 | SET
```

Starts with db service and creates the database

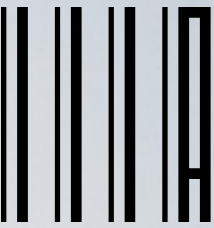


Running

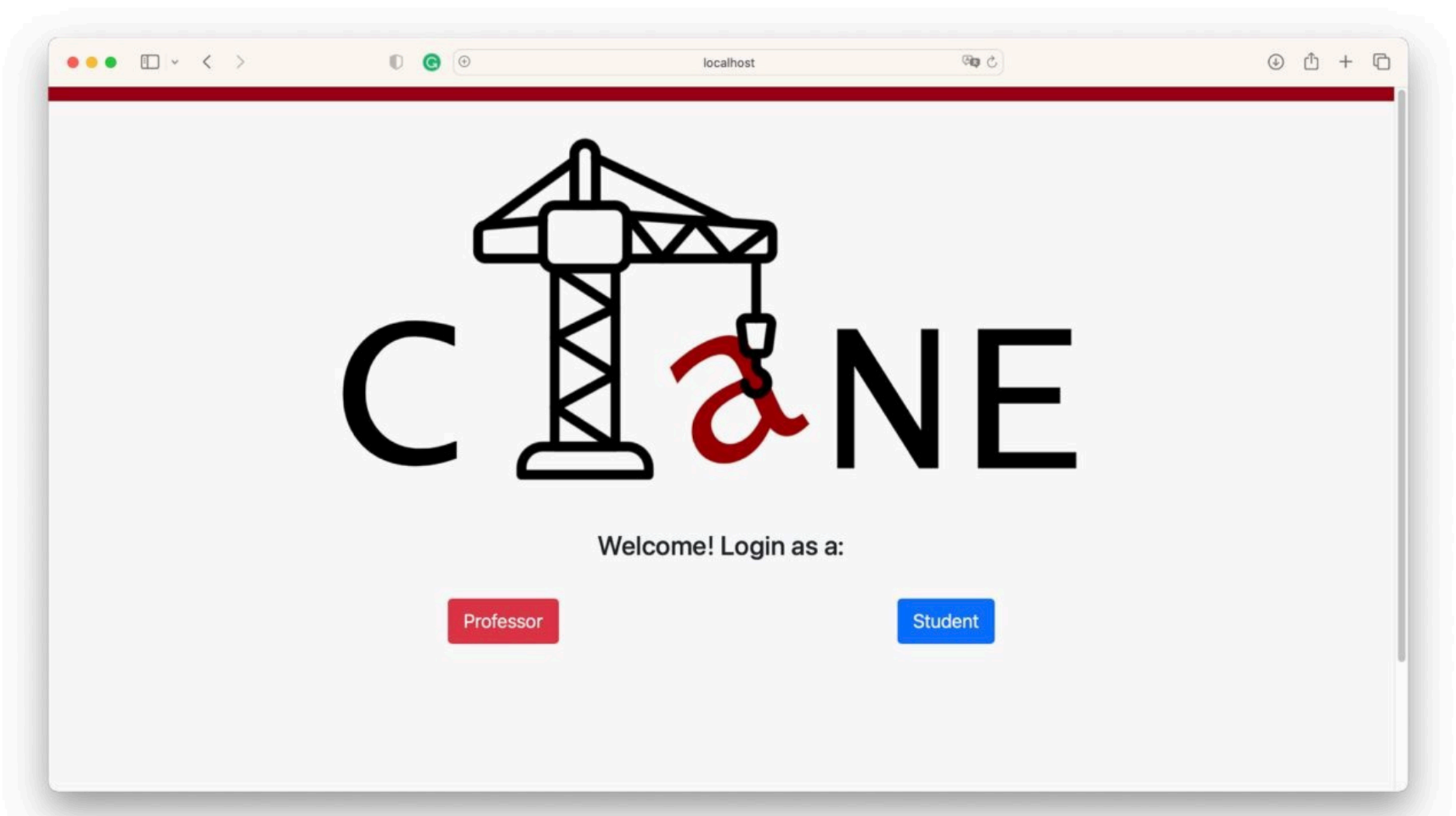


```
docker-db-1 | ALTER TABLE
docker-db-1 | ALTER TABLE
docker-db-1 | ALTER TABLE
docker-db-1 | ALTER TABLE
docker-db-1 | ALTER TABLE
docker-db-1 | ALTER TABLE
docker-db-1 | ALTER TABLE
docker-db-1 | ALTER TABLE
docker-db-1 |
docker-db-1 |
docker-db-1 | 2023-05-30 09:33:12.796 UTC [84] LOG:  received fast shutdown request
docker-db-1 | waiting for server to shut down...2023-05-30 09:33:12.800 UTC [84] LOG:  aborting any active transactions
docker-db-1 | 2023-05-30 09:33:12.819 UTC [84] LOG:  background worker "logical replication launcher" (PID 90) exited with exit code 1
docker-db-1 | 2023-05-30 09:33:12.820 UTC [85] LOG:  shutting down
docker-db-1 | 2023-05-30 09:33:12.825 UTC [85] LOG:  checkpoint starting: shutdown immediate
docker-db-1 | .2023-05-30 09:33:14.830 UTC [85] LOG:  checkpoint complete: wrote 946 buffers (5.8%); 0 WAL file(s) added, 0 removed, 0 recycled; write=1.97
4 s, sync=0.005 s, total=2.010 s; sync files=260, longest=0.003 s, average=0.001 s; distance=4444 kB, estimate=4444 kB
docker-db-1 | .2023-05-30 09:33:14.919 UTC [84] LOG:  database system is shut down
docker-db-1 | done
docker-db-1 | server stopped
docker-db-1 |
docker-db-1 | PostgreSQL init process complete; ready for start up.
docker-db-1 |
docker-db-1 | 2023-05-30 09:33:15.089 UTC [1] LOG:  starting PostgreSQL 15.3 (Debian 15.3-1.pgdg110+1) on x86_64-pc-linux-gnu, compiled by gcc (Debian 10.2
.1-6) 10.2.1 20210110, 64-bit
docker-db-1 | 2023-05-30 09:33:15.090 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
docker-db-1 | 2023-05-30 09:33:15.090 UTC [1] LOG:  listening on IPv6 address ":::", port 5432
docker-db-1 | 2023-05-30 09:33:15.098 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
docker-db-1 | 2023-05-30 09:33:15.212 UTC [100] LOG:  database system was shut down at 2023-05-30 09:33:14 UTC
docker-db-1 | 2023-05-30 09:33:15.302 UTC [1] LOG:  database system is ready to accept connections
docker-web-1 | 30-May-2023 09:33:15.447 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Server version name: Apache Tomcat/10.1.9
docker-web-1 | 30-May-2023 09:33:15.456 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Server built: May 9 2023 14:30:05 UTC
docker-web-1 | 30-May-2023 09:33:15.459 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Server version number: 10.1.9.0
docker-web-1 | 30-May-2023 09:33:15.460 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log OS Name: Linux
docker-web-1 | 30-May-2023 09:33:15.461 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log OS Version: 5.15.49-linuxkit
docker-web-1 | 30-May-2023 09:33:15.466 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Architecture: amd64
docker-web-1 | 30-May-2023 09:33:15.466 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Java Home: /opt/java/openjdk
docker-web-1 | 30-May-2023 09:33:15.472 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log JVM Version: 17.0.7+7
docker-web-1 | 30-May-2023 09:33:15.473 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log JVM Vendor: Eclipse Adoptium
docker-web-1 | 30-May-2023 09:33:15.473 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log CATALINA_BASE: /usr/local/tomcat
docker-web-1 | 30-May-2023 09:33:15.474 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log CATALINA_HOME: /usr/local/tomcat
docker-web-1 | 30-May-2023 09:33:15.503 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Djava.util.logging.config.
file=/usr/local/tomcat/conf/logging.properties
docker-web-1 | 30-May-2023 09:33:15.504 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Djava.util.logging.manager
=org.apache.juli.ClassLoaderLogManager
docker-web-1 | 30-May-2023 09:33:15.506 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Djdk.tls.ephemeralDHKeySiz
e=2048
docker-web-1 | 30-May-2023 09:33:15.507 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Djava.protocol.handler.pkg
```

Running



Connecting to `localhost:8080/crane` we can interact with the web application





How to...



- To containerize your group project you have to:
 - Install docker on your machine;
 - Modify the docker-compose.yml file provided for crane project accordingly;
 - Place this file in the same folder of your war file generated with maven;
- You have to install docker and docker compose following the guidelines in the documentation
- Docker: <https://docs.docker.com/get-docker/>
- Docker compose: <https://docs.docker.com/compose/>



How to...



- To manage, run, stop, remove your container and have a clear overview of what is going on you can download Docker Desktop.
- Open a new terminal, place in the same folder of your docker-compose.yaml file and check that the docker daemon is alive. Then type:
 - `docker-compose up` to create and run a new container
 - `docker-compose down` to stop the container (your web app won't be visible anymore)
 - `docker ps` to list the running containers
 - `docker ps -a` to list the containers — also those not running anymore



How to...



- To interact with the database:
 - `docker ps` to list the running containers. Then, select the name of the Postgres container in my case `docker-db-1`
 - `Docker exec docker-db-1 psql -U postgres` to access the psql command line